

bomblab 报告

解题报告

phase_1

phase_2

phase_3

phase_4

phase_5

phase_6

secret_phase

反馈/收获/感悟/总结

bomblab 报告

姓名:刘远航

学号：2023202275

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	1	1	1	1	1	1

scoreboard 截图：

2023202275	0	0	0	0	0	1	0	0	0	0	0	0	0	1	7
------------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

解题报告

phase_1

1 | Can you see the meaning inside yourself? Can you see the meaning in your true destiny?

思路分析：

1. 汇编代码解读：
- phase_1 函数首先通过 `lea 0x1d40(%rip), %rsi` 指令将一个预设的字符串地址（位于 `0x3180`）加载到 `%rsi` 寄存器中。

`%rdi` 寄存器中存放的是用户输入的字符串地址。

接着调用 `strings_not_equal` 函数进行比较。

如果返回值 `%eax` 为 0（表示两字符串相等），则函数正常返回；否则，调用 `explode_bomb` 引爆炸弹。
2. 提取目标字符串：
- GDB 中调用x/s查看地址 `0x3180` 处的内容，可以得到目标字符串为：`"Can you see the meaning inside yourself? Can you see the meaning in your true destiny?"`。
3. 伪代码：

```

1 void phase_1(char *input) {
2     char *target = "Can you see the meaning inside yourself? Can you see the
    meaning in your true destiny?";
3     if (strings_not_equal(input, target) != 0) {
4         explode_bomb();
5     }
6 }

```

phase_2

```
1 | 659792 1012670 349543 606734
```

思路分析：

1. 输入：

- 通过汇编代码可以看出，`phase_2` 首先调用了 `sscanf`，从输入字符串中读取了 4 个整数。这些整数被存储在栈上，起始地址为 `%rsp`。
- `cmp $0x4, %eax` 检查 `sscanf` 的返回值是否为 4，如果不是则引爆炸弹。

2. 一段繁琐矩阵乘法，但实际上不用细看，不过因为报告要求所以我在此给出完整流程：

- 代码段 `148e` 到 `1500` 包含了一个三重嵌套循环，执行的是两个矩阵的乘法操作。
- 矩阵 A (matA)：** 位于 `0x6140`，大小为 2×3 。外层循环 `%r11d` 遍历行（2 行），每次步进 `0xc`（3 个整数）。
- 矩阵 B (matB)：** 位于 `0x6120`，大小为 3×2 。内层循环 `%r8d` 遍历列（2 列）。
- 乘法过程：** 最内层循环 `%rax` 计算点积 $\sum_{k=0}^2 A_{ik} \times B_{kj}$ 。注意 `matB` 的寻址 (`%rsi, %rax, 8`) 表明矩阵 B 在内存中是以行优先存储的，跨行步进为 8 字节（即 2 个整数）。
- 计算结果（一个 2×2 的矩阵，共 4 个整数）被存储在 `%rsp + 0x10` 开始的内存区域中。

3. 比较：

- 在计算完成后，代码进入一个循环（`150c` 到 `151f`），将计算出的 4 个结果（从 `%rsp + 0x10` 开始）与用户的输入（从 `%rsp` 开始）逐一进行比较。
- 只有当我输入的 4 个数字与矩阵乘法的结果完全一致时，才能通过此关。

4. 伪代码：

```

1 void phase_2(char *input_str) {
2     int input[4];
3     if (sscanf(input_str, "%d %d %d %d", &input[0], &input[1], &input[2],
    &input[3]) != 4) {
4         explode_bomb();
5     }
6
7     int matA[2][3] = { /* 内存 0x6140 处的值 */ };
8     int matB[3][2] = { /* 内存 0x6120 处的值 */ };
9     int result[4];
10

```

```

11     for (int i = 0; i < 2; i++) {
12         for (int j = 0; j < 2; j++) {
13             int sum = 0;
14             for (int k = 0; k < 3; k++) {
15                 sum += matA[i][k] * matB[k][j];
16             }
17             result[i * 2 + j] = sum;
18         }
19     }
20
21     for (int i = 0; i < 4; i++) {
22         if (input[i] != result[i]) {
23             explode_bomb();
24         }
25     }
26 }

```

phase_3

1 | 1 u 535

思路分析：

phase_3、4拆的时候感觉基本都不用看懂代码在干嘛，跟着汇编走就可以挖到答案了。

1. 输入：

- phase_3 调用 sscanf 读取三个值，根据参数地址推断格式为 "%d %c %d"。
- 第一个整数（索引）存储在 0x10(%rsp)，字符存储在 0xf(%rsp)，第二个整数存储在 0x14(%rsp)。

2. 异或预处理：

- 指令 xor %al, 0xf(%rsp) 将输入的字符与内存 0x6110 处的 mask.1 (0x20) 进行异或。这实际上是在切换字母的大小写。

3. 跳转表逻辑：

- 程序检查第一个整数是否大于 7，若大于则引爆炸弹。
- 使用跳转表（位于 0x3240）根据第一个整数进行跳。

4. 分支匹配（我选择的是1）：

- 跳转到 15ca 分支：将 %eax 设置为 0x75（字符 'u'），并比较第二个整数是否等于 0x217（十进制 535）。
- 最后在 16a3 处比较 0xf(%rsp)（异或后的输入字符）是否等于 %al（即 'u'）。
- 因为 'u' ^ 0x20 == 'u'，所以最终确定的输入为 1 u 535。

5. 伪代码：

```

1 void phase_3(char *input_str) {
2     int idx, val;
3     char c;
4     if (sscanf(input_str, "%d %c %d", &idx, &c, &val) <= 2) explode_bomb();
5 }

```

```

6      c ^= 0x20; // mask.1
7      if (idx > 7) explode_bomb();
8
9      switch (idx) {
10         case 1:
11             if (val != 535) explode_bomb();
12             if (c != 'u') explode_bomb();
13             break;
14             // ... 其他分支 ...
15         default: explode_bomb();
16     }
17 }

```

phase_4

1 | 31 AB

思路分析:

1. 输入:

- phase_4 调用 sscanf 读取一个整数 (存储在 0xc(%rsp)) 和一个字符串 (存储在 0x10(%rsp))。

2. 整数部分 (func4_1) :

- 程序调用 func4_1(5)。
- func4_1 是一个递归函数, 逻辑为:
 - 若 $n \leq 0$ 返回 0; 若 $n = 1$ 返回 1。
 - 否则返回 $2 \times func4_1(n - 1) + 1$
- $f(1) = 1, f(2) = 3, f(3) = 7, f(4) = 15, f(5) = 31$ 。因此, 输入的第二个参数必须是 31。

3. 字符串部分 (func4_2) :

- 程序调用 func4_2(5, 2, 'A', 'C', 'B', buf) 生成一个字符串。
- 该函数内部逻辑较为复杂, 包含多个递归调用。通过在 GDB 中追踪其递归过程, 发现它最终会在缓冲区中按顺序写入字符。
- 在 $n = 5, k = 2$ 的初始状态下, 经过递归路径选择, 最终写入的字符序列为 "AB"。

4. 伪代码:

```

1  int func4_1(int n) {
2      if (n <= 0) return 0;
3      if (n == 1) return 1;
4      return 2 * func4_1(n - 1) + 1;
5  }
6
7  void phase_4(char *input_str) {
8      int val;
9      char s[10];
10     if (sscanf(input_str, "%d %s", &val, s) != 2) explode_bomb();
11     if (val != func4_1(5)) explode_bomb();

```

```
12     char buf[3];
13     func4_2(5, 2, 'A', 'C', 'B', buf);
14     if (strings_not_equal(s, buf)) explode_bomb();
15 }
```

phase_5

```
1 | 111127
```

思路分析：

1. 长度检查：

- phase_5 首先通过 `string_length` 检查输入字符串的长度，必须为 6 个字符，否则引爆炸弹。

2. 循环逻辑：

- 程序遍历这 6 个字符，对每个字符执行以下操作：
 - `movzb1 (%rax), %edx`：取出当前字符。
 - `and $0xf, %edx`：取该字符的低 4 位（即 `char & 0x0F`）。
 - `add (%rsi, %rdx, 4), %ecx`：以该低 4 位作为索引，从一个预定义的整数数组（位于 `0x3260`）中取出对应的整数，并累加到寄存器 `%ecx` 中。

3. 验证：

- 循环结束后，比较累加和 `%ecx` 是否等于 `49`。

4. 解题过程：

- 通过 `x/wd` 查看 `0x3260` 处的数组内容，得到如下 16 个整数：
`[2, 10, 6, 1, 12, 16, 9, 3, 4, 7, 14, 5, 11, 8, 15, 13]`
- 我们需要找到 6 个索引，使其对应的数组值之和为 49。
- 我的输入为 `111127`，对应的索引序列为 `[1, 1, 1, 1, 2, 7]`。

5. 伪代码：

```
1 void phase_5(char *input) {
2     if (strlen(input) != 6) explode_bomb();
3     int sum = 0;
4     int array[16] = {2, 10, 6, 1, 12, 16, 9, 3, 4, 7, 14, 5, 11, 8, 15, 13};
5     for (int i = 0; i < 6; i++) {
6         int idx = input[i] & 0x0F;
7         sum += array[idx];
8     }
9     if (sum != 49) explode_bomb();
10 }
```

phase_6

1 | 1 4 6 5 2 3

思路分析:

1. 输入:

- `phase_6` 首先调用 `read_six_numbers` 读取 6 个整数。
- 随后进行两项检查: 每个数字必须在 1 到 6 之间, 且 6 个数字互不相同。

2. 数值转换:

- 在 1906 到 1917 的循环中, 程序将每个输入值 x 替换为 $7 - x$ 。例如, 如果输入是 1, 则变为 6。

3. 链表重排:

- 程序中存在一个链表 (起始于 `node1`, 地址 `0x6220`)。每个节点包含一个整数值和指向下一个节点的指针。
- 程序根据转换后的 6 个数字作为索引, 从原始链表中按顺序挑选节点, 并重新连接成一个新的链表。

4. 排序验证:

- 最后的循环 (19c3 到 19cb) 遍历重排后的链表, 检查节点的数值是否满足降序排列。如果不满足则引爆炸弹。

5. 解题过程:

- 通过内存查看, 原始链表各节点的值为:
 - Node 1: 661, Node 2: 605, Node 3: 848, Node 4: 187, Node 5: 226, Node 6: 949。
- 降序排列的值顺序为: 949 (Node 6) > 848 (Node 3) > 661 (Node 1) > 605 (Node 2) > 226 (Node 5) > 187 (Node 4)。
- 因此, 转换后的序列应为: 6 3 1 2 5 4。
- 最终答案为: 1 4 6 5 2 3。

6. 伪代码:

```
1 void phase_6(char *input) {
2     int nums[6];
3     read_six_numbers(input, nums);
4     for (int i = 0; i < 6; i++) {
5         if (nums[i] < 1 || nums[i] > 6) explode_bomb();
6         for (int j = i + 1; j < 6; j++) {
7             if (nums[i] == nums[j]) explode_bomb();
8         }
9     }
10    for (int i = 0; i < 6; i++) nums[i] = 7 - nums[i];
11
12    Node *nodes[6];
13    for (int i = 0; i < 6; i++) {
14        Node *curr = &node1;
15        for (int j = 1; j < nums[i]; j++) curr = curr->next;
16        nodes[i] = curr;
17    }
```

```

18     for (int i = 0; i < 5; i++) {
19         if (nodes[i]->value < nodes[i+1]->value) explode_bomb();
20     }
21 }

```

secret_phase

1 | cchbb

思路分析：

1. 入口触发：

- `secret_phase` 隐藏在 `phase_defused` 函数中。当程序检测到 `phase_6` 的输入字符串末尾带有关键词 `"unlock"` 时，会触发此隐藏阶段。
- 解密入口：在 `phase_6` 的答案后加上 `" unlock"`。

2. func7：

- **核心数据结构**：程序定义了一个 8×8 的迷宫（由 `row0` 到 `row7` 组成），其中 `1` 表示障碍物，`0` 表示通路。
- **状态维护**：程序在递归函数 `func7` 中维护当前的坐标 (r, c) ，初始位置为 $(0, 0)$ 。
- **移动规则**：
 - 输入字符串的每个字符决定一步移动。
 - 使用字符的**低 3 位**（`char & 0x7`）作为索引，从一个包含 8 种偏移量的预定义表中读取移动步长。
 - 每一步移动包含两部分检查：目标位置是否在迷宫内且非障碍物。
- 当坐标到达目标点 $(4, 7)$ 且字符串处理完毕时，递归返回 0，任务达成。

3. 解题过程：

- 先通过链表结构得到如下 8×8 的地图：

	0	1	2	3	4	5	6	7
Row 0	0	0	1	0	0	1	0	0
Row 1	0	0	0	1	0	0	0	1
Row 2	1	0	1	0	0	1	0	0
Row 3	1	0	0	0	0	0	0	0
Row 4	0	1	0	0	1	0	1	0
Row 5	1	0	0	1	1	0	0	0
Row 6	0	0	0	0	0	1	0	1
Row 7	0	1	0	0	0	0	0	0

根据地图规划一条路径即可，最后得到 `cchbb`。

4. 伪代码：

```
1  int road_map[8][8] = { /* 地图如上 */ };
2  int moves[8][2] = { {-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2}, {-1, -2},
3                          {-2, -1} };
4
5  int func7(char *path, int r, int c, int step) {
6      if (r == 4 && c == 7) return 0; // 终点
7      if (path[step] == '\0') return 1;
8
9      int move_idx = path[step] & 0x7;
10     int next_r = r + moves[move_idx][0];
11     int next_c = c + moves[move_idx][1];
12
13     if (isValid(next_r, next_c) && !isBlocked(next_r, next_c)) {
14         return func7(path, next_r, next_c, step + 1);
15     }
16     return 1; // 撞墙或出界
17 }
```

反馈/收获/感悟/总结

整个lab加在一起可能花了我共1~2day的样子，但是因为中间各种各样的事情，我花了时间跨度3周的样子完成它。

我觉得从底层去还原一个代码的逻辑是一件非常有意思的逆向工程，尤其是以拆弹这种方式。bomblab让我看汇编的速度和反应快了很多。

整个拆解过程中，我觉得最重要的还是耐心吧，si一行一行地看，重点关注explode_bomb前的跳转代码。中间的几个phase（比如2,3,4）其实都不用对代码有一个完整的理解，从phase_6开始，需要对每个循环部分看懂他的意图是什么，secret_phase几乎占据了我一半的时间，找到入口后还有很远的路要走。

总之，这是我目前做过最爽的lab了。不过，我似乎不是很认可爆炸扣分这个行为的必要性？因为大家都知道在爆炸处设置断点（虽然我因为手贱爆了一个hh）。